

SERVERLESS ARCHITECTURE USING AWS FOR INVENTORY MANAGEMENT

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. Anil Kumar Varanasi, APSSDC

Submitted by

GAJJALA LOKESWAR REDDY, S200714, RGUKT SRIKAKULAM, (Batch -2)

BUSETTY GURU CHAITANYA VARDHAN, S200720, RGUKT SRIKAKULAM, (Batch -2)

PASUPULA JYOTHI KUMAR REDDY, S200748, RGUKT SRIKAKULAM, (Batch -2)

N MANJUNATH, S00727, RGUKT SRIKAKULAM, (Batch -2)

M YOGA SUSMITHA, S200718, RGUKT SRIKAKULAM, (Batch -2)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr Anil Kumar Varanasi, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Chennu DivyaKrishna, 24BQ5A0504, divyakrishna8779@gmail.com.

Gamini Maheswari Sri, 24BQ5A0505, gaminimaheswari@gmail.com.

Kalinta Shalini, 24BQ5A0507, kalintashalini@gmail.com.

Jonnalagadda Sai Prasad, 24BQ5A0506, jsaiprasadcse@gmail.com.

Puvvula Sai Surya Rohith, 24BQ5A0529, saisuryarohithp2004@gmail.com.

ABSTRACT

This project presents the implementation of **CloudCart**, a fully serverless e-commerce platform using Amazon Web Services (AWS) to provide a secure, scalable, and highly available online shopping experience. The primary goal is to leverage serverless computing to build a modern e-commerce application that eliminates infrastructure management while ensuring seamless user interaction, automatic scalability, and cost-efficient resource utilization. The system workflow begins when users access the web application hosted on Amazon S3 and delivered globally through Amazon CloudFront. Users can securely register and log in using Amazon Cognito, which manages authentication and authorization. After successful authentication, users can browse products stored in Amazon DynamoDB, add products to their shopping cart, and place orders through a user-friendly interface. The frontend communicates with backend services via Amazon API Gateway, which invokes AWS Lambda functions to process user requests such as retrieving product details, managing shopping carts, and handling order placement. The Lambda functions interact with DynamoDB to perform real-time data storage and retrieval, ensuring fast and reliable access to application data.

This architecture is fully serverless, utilizing Amazon S3 for static website hosting, Amazon CloudFront for secure and low-latency content delivery, Amazon Cognito for user authentication, Amazon API Gateway for RESTful API management, AWS Lambda for backend business logic, DynamoDB for NoSQL data storage, and IAM for secure access control and service permissions. The solution provides automatic scaling, high availability, reduced operational complexity, and a pay-as-you-use pricing model without requiring traditional server administration.

By dynamically processing user requests and integrating managed AWS services, the platform delivers a responsive, secure, and efficient online shopping experience. Users can perform essential e-commerce operations such as account management, product browsing, shopping cart management, and online order placement with minimal latency and maximum reliability. The project demonstrates how serverless cloud architecture can be effectively applied to develop real-world e-commerce solutions, providing a strong foundation for future enhancements such as payment gateway integration, order tracking, recommendation systems, inventory management, analytics dashboards, and AI-powered personalized shopping experiences.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	INTRODUCTION.....	5
2.	METHODOLOGY	6
3.	TECHNOLOGY STACK	9
4.	SYSTEM DESIGN / ARCHITECTURE	11
5.	IMPLEMENTATION.....	23
6.	RESULTS	26
7.	CONCLUSION	29

1. INTRODUCTION

In the rapidly evolving digital economy, where online shopping has become an integral part of everyday life, businesses require e-commerce platforms that are scalable, secure, and capable of delivering seamless customer experiences. Traditional e-commerce applications often rely on dedicated servers that demand continuous infrastructure provisioning, maintenance, monitoring, and scaling. These challenges increase operational costs and limit the ability of businesses to respond efficiently to fluctuating customer demands. This project, titled **CloudCart: A Fully Serverless E-Commerce Platform**, leverages the capabilities of Amazon Web Services (AWS) to build a modern cloud-native solution that eliminates the complexities of server management while providing high availability, automatic scalability, and enhanced security.

With the growing adoption of cloud computing, serverless architectures have transformed the way web applications are designed and deployed. Instead of managing physical or virtual servers, developers can focus entirely on application development while AWS automatically handles infrastructure management, resource allocation, fault tolerance, and scaling. CloudCart embraces this serverless paradigm by integrating multiple AWS managed services to create a reliable, efficient, and event-driven e-commerce platform capable of supporting real-world online shopping requirements.

Designed with scalability, security, and user experience as its primary objectives, CloudCart enables customers to securely register and log in through Amazon Cognito, browse products stored in Amazon DynamoDB, add items to their shopping cart, and place orders through an intuitive web interface. The frontend application is hosted on Amazon S3 and globally distributed using Amazon CloudFront, ensuring low-latency access and improved performance for users across different geographical regions. User requests are securely routed through Amazon API Gateway, which invokes AWS Lambda functions to execute backend business logic such as retrieving product information, managing shopping carts, validating user requests, and processing customer orders. These Lambda functions interact seamlessly with DynamoDB to perform real-time data storage and retrieval, ensuring consistent performance and efficient database operations.

One of the key strengths of CloudCart lies in its fully serverless architecture, which removes the need for server provisioning, operating system maintenance, capacity planning, or manual scaling. Every AWS service operates independently and responds dynamically to user requests, allowing the platform to automatically scale based on application traffic while optimizing resource utilization and reducing operational expenses. Security is strengthened through AWS Identity and Access Management (IAM), which enforces fine-grained access control between AWS services, while Amazon CloudWatch enables continuous monitoring, logging, and performance analysis to maintain application reliability and quickly identify operational issues.

This report presents a comprehensive overview of the CloudCart system, including its architecture, design principles, implementation methodology, and integration of various AWS services. It explains how serverless technologies collaborate to deliver a secure, responsive, and cost-effective e-commerce platform while demonstrating best practices in cloud-native application development. The modular architecture of CloudCart also provides flexibility for future enhancements, including secure payment gateway integration, inventory management, order tracking, recommendation engines powered by artificial intelligence, customer analytics

dashboards, and advanced business intelligence solutions.

By delivering a scalable, event-driven, and fully serverless e-commerce platform, CloudCart demonstrates the practical application of modern cloud computing technologies in addressing real-world business challenges. The project showcases how AWS serverless services can be effectively utilized to build intelligent, highly available, and cost-efficient online shopping solutions that meet the growing demands of digital commerce while providing a strong foundation for future innovation and expansion.

2. METHODOLOGY

The development of **CloudCart** follows a cloud-native, serverless methodology using Amazon Web Services (AWS). The system is designed to provide a secure, scalable, and highly available e-commerce platform by integrating multiple AWS managed services. Instead of relying on traditional server-based infrastructure, the application adopts an event-driven architecture where each AWS service performs a specific task and communicates seamlessly with other services. This approach minimizes operational overhead, enables automatic scaling, and ensures efficient resource utilization.

The implementation process begins with the deployment of the frontend application on **Amazon S3**, where static website files such as HTML, CSS, JavaScript, or React components are stored. To improve website performance and reduce latency, **Amazon CloudFront** is configured as a Content Delivery Network (CDN) that caches and distributes the frontend content across global edge locations. This enables users to access the application quickly regardless of their geographical location while ensuring secure content delivery through HTTPS.

User authentication is implemented using **Amazon Cognito**, which provides secure user registration, login, and session management. Cognito handles user identity verification, password management, and authentication tokens without requiring developers to build or maintain custom authentication mechanisms. After successful authentication, users receive secure access tokens that are used to access protected backend APIs.

The frontend communicates with backend services through **Amazon API Gateway**, which acts as the secure entry point for all client requests. API Gateway exposes RESTful APIs for operations such as viewing products, managing shopping carts, retrieving user information, and placing customer orders. It validates incoming requests, manages API routing, and securely invokes the corresponding AWS Lambda functions.

The application logic is implemented using **AWS Lambda**, which executes backend functions on demand without requiring dedicated servers. Separate Lambda functions are developed for handling different business operations, including retrieving product information, adding items to the shopping cart, updating cart contents, processing customer orders, and fetching order history. Lambda automatically scales based on incoming requests, ensuring consistent application performance during varying traffic loads while optimizing computing costs.

All application data is stored in **Amazon DynamoDB**, a fully managed NoSQL database that provides low-latency data access and automatic scaling. Product information, customer details, shopping cart data, and order records are stored as separate items within DynamoDB tables. Lambda functions perform Create, Read, Update, and Delete (CRUD) operations on the database in real time, ensuring accurate and consistent data management throughout the application lifecycle.

Security is enforced through **AWS Identity and Access Management (IAM)** by assigning least-privilege permissions to every AWS service. IAM roles allow Lambda functions to securely access DynamoDB, API Gateway, Cognito, and other AWS resources without exposing sensitive credentials. This ensures secure communication between services while protecting application data from unauthorized access.

System monitoring and operational logging are performed using **Amazon CloudWatch**. CloudWatch continuously collects logs, execution metrics, API performance statistics, and Lambda invocation details. These monitoring capabilities enable administrators to identify errors, analyze application performance, and troubleshoot operational issues efficiently.

The overall workflow begins when a user accesses the CloudCart website through CloudFront. The frontend is loaded from Amazon S3, after which users register or log in using Amazon Cognito. Once authenticated, users browse available products, add desired items to the shopping cart, and place orders. Every user request is routed through API Gateway, processed by AWS Lambda, and stored or retrieved from DynamoDB. The response is then returned to the frontend, providing users with a seamless and responsive shopping experience.

TECHNOLOGY STACK

CloudCart is developed using a fully serverless architecture on **Amazon Web Services (AWS)**. The project integrates multiple AWS managed services to provide secure authentication, scalable backend processing, reliable data storage, and efficient content delivery. The following technologies and services were used during the development of the application.

1.1. AWS Services

Amazon S3:

Hosts the static frontend files (HTML, CSS, JavaScript/React) and serves the web application.

AWS Lambda:

Executes backend business logic such as product retrieval, shopping cart management, and order processing without managing servers.

Amazon DynamoDB:

Stores product information, user data, shopping cart details, and customer orders in a highly scalable NoSQL database.

Amazon Cognito:

Provides secure user registration, login, authentication, and user management.

Amazon CloudFront:

Delivers frontend content globally with low latency and enhanced security through Content Delivery Network (CDN).

Amazon API Gateway:

Creates secure REST APIs and acts as the communication bridge between the frontend and backend services.

AWS Identity and Access Management (IAM):

Manages authentication, authorization, and secure permissions between AWS services.

Amazon CloudWatch:

Monitors application performance, stores logs, and tracks Lambda execution and API activity.

1.1. Programming & Scripting

- **HTML5:** Structures the user interface of the web application.
- **CSS3:** Designs and styles the frontend interface for better user experience
- **JavaScript:** Implements client-side functionality and interacts with backend APIs.

The combination of these technologies enables CloudCart to provide a secure, scalable, cost-effective, and highly available e-commerce platform. By leveraging AWS serverless services, the application eliminates infrastructure management while ensuring automatic scaling, fault

SYSTEM DESIGN / ARCHITECTURE

The **CloudCart** system is designed using a **fully serverless architecture** on **Amazon Web Services (AWS)**. The architecture integrates multiple AWS managed services to provide a secure, scalable, highly available, and cost-effective e-commerce platform. Each service performs a specific responsibility, enabling efficient communication between the frontend, backend, authentication system, and database while eliminating the need for traditional server management.

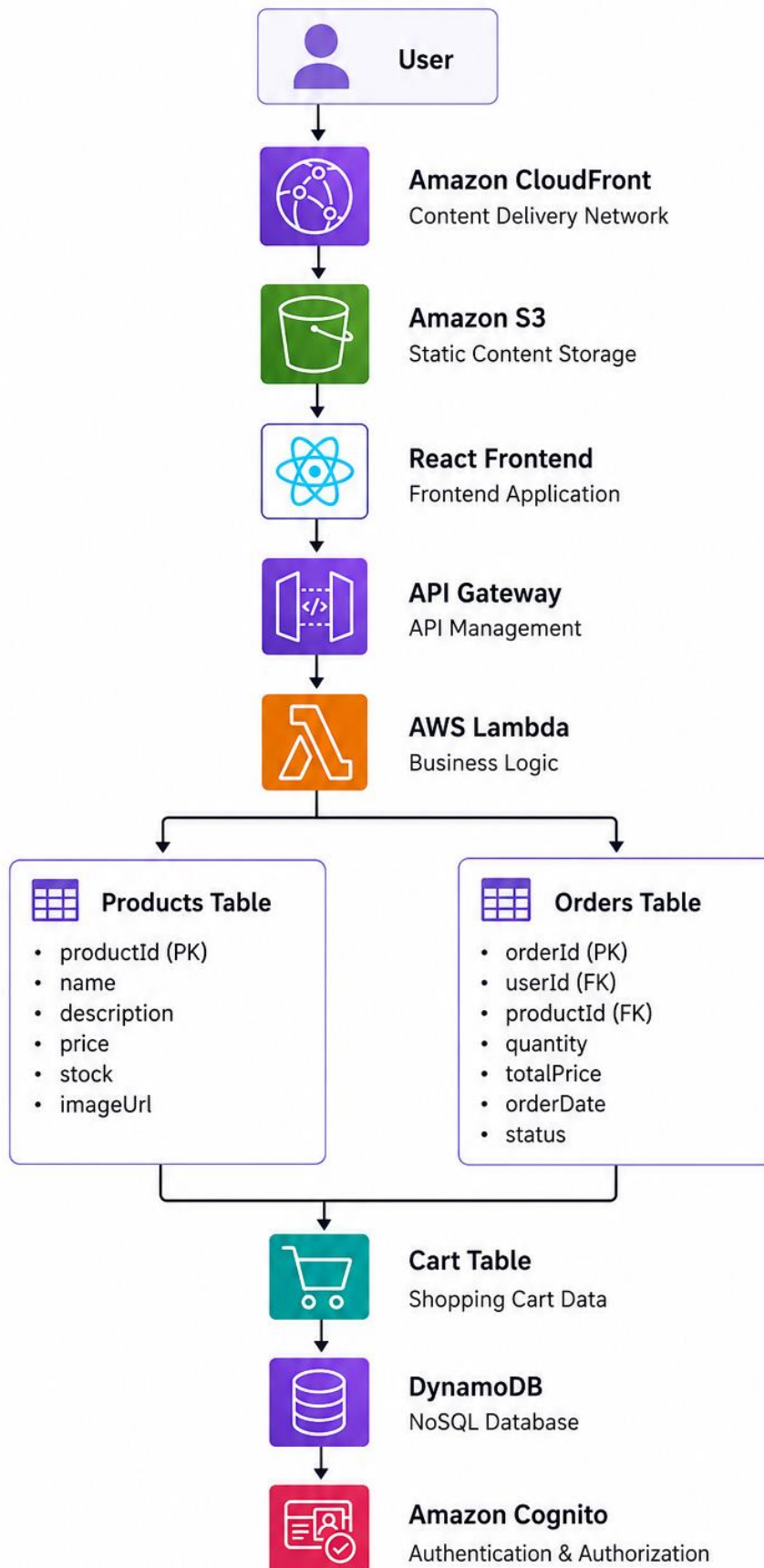
The application workflow begins when a user accesses the CloudCart website through **Amazon CloudFront**, which acts as a global Content Delivery Network (CDN). CloudFront delivers the frontend content with low latency by retrieving static website files stored in **Amazon S3**. The frontend application, developed using **React**, provides an interactive interface that allows users to browse products, manage their shopping cart, and place orders.

For secure access, users register and authenticate using **Amazon Cognito**. Cognito manages user identities, authentication, and authorization by issuing secure access tokens after successful login. These tokens are used to access protected backend APIs, ensuring that only authenticated users can perform authorized operations within the application.

Whenever the frontend sends a request, it is routed through **Amazon API Gateway**, which serves as the central API management layer. API Gateway securely receives client requests and forwards them to the appropriate **AWS Lambda** function. Lambda executes the required business logic, including retrieving product details, managing shopping cart operations, validating user requests, and processing customer orders.

Application data is stored in **Amazon DynamoDB**, a fully managed NoSQL database that provides high-speed data access and automatic scalability. The database maintains three primary logical tables: the **Products Table**, **Orders Table**, and **Cart Table**. The Products Table stores product information such as product ID, name, description, price, stock quantity, and image URL. The Orders Table maintains customer order details including order ID, user ID, product ID, quantity, total price, order date, and order status. The Cart Table temporarily stores shopping cart information before the order is finalized.

SYSTEM ARCHITECTURE



AWS **Identity and Access Management (IAM)** is used to securely control permissions between AWS services. IAM roles ensure that Lambda functions, API Gateway, DynamoDB, and Cognito communicate securely using least-privilege access principles without exposing sensitive credentials.

The architecture follows an **event-driven serverless model**, where AWS Lambda functions execute only when API requests are received. This enables automatic scaling based on user traffic, minimizes infrastructure costs, and improves overall application performance. Since all backend components are fully managed by AWS, the platform provides high availability, fault tolerance, and reduced operational complexity.

Overall, the CloudCart architecture demonstrates how multiple AWS serverless services can be integrated to build a modern cloud-native e-commerce platform. The modular design simplifies maintenance, improves scalability, and provides flexibility for future enhancements such as payment gateway integration, inventory management, AI-powered recommendation systems, analytics dashboards, and order tracking services.

1.2. Data Ingestion Layer – Amazon S3

The process begins when a user uploads a **CSV file** containing product details (e.g., Product Name, Quantity) to an **Amazon S3 bucket**. S3 acts as the storage layer for input files. An **S3 event trigger** is configured to detect new object uploads and invoke the processing component automatically.

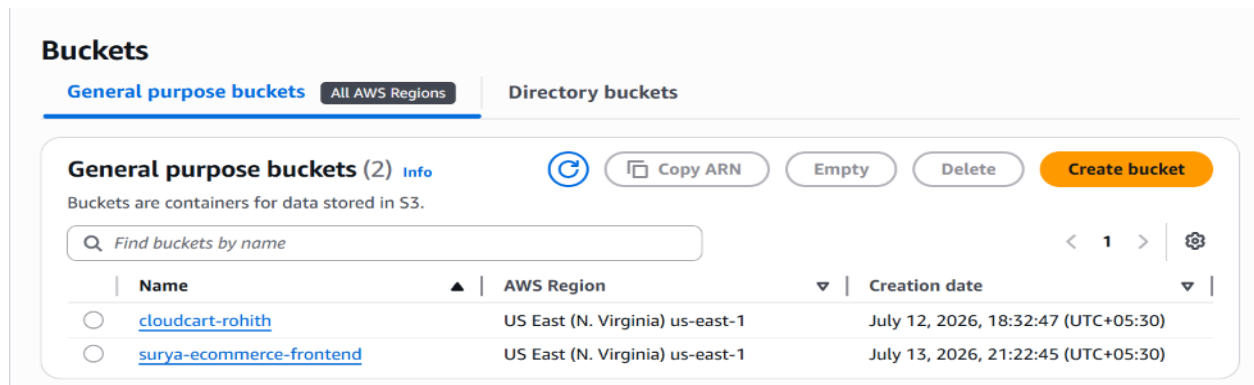


Figure 1.bucket name

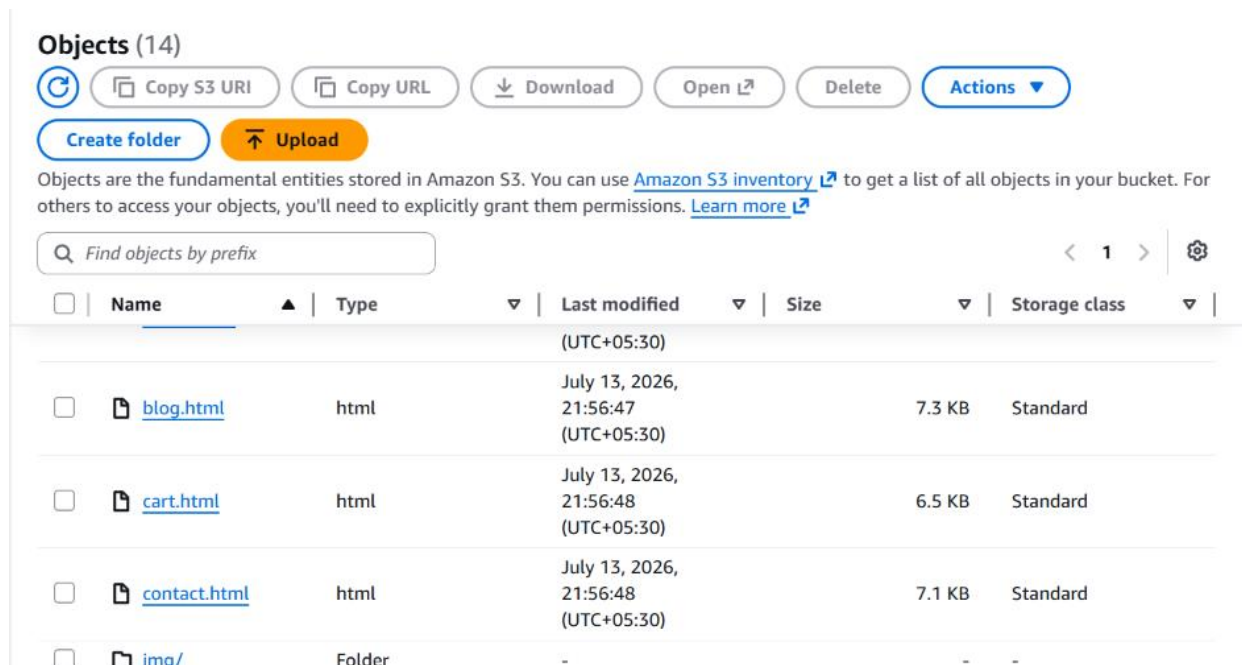


Figure 2.csv file uploaded into s3 bucket

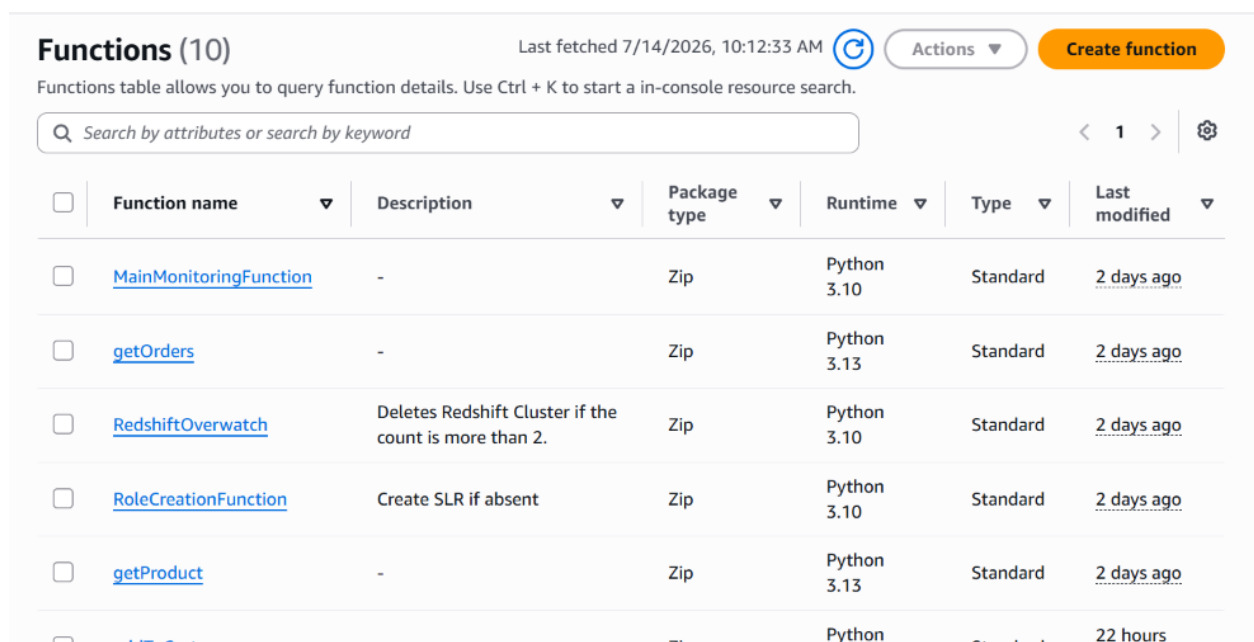
Key Features :

- Supports various file formats (e.g., .csv)
- Triggers Lambda functions upon file upload
- Scalable and cost-effective storage

1.3. Processing Logic – AWS Lambda

An **AWS Lambda function** is triggered by the S3 event. This function performs multiple tasks:

- Reads and parses the CSV file
- Validates product and quantity data
- Uploads each product record into **Amazon DynamoDB**
- Scans the table to identify products below a minimum quantity threshold
- Sends an email summary of low-stock items via **Amazon SNS**



The screenshot shows the AWS Lambda console interface. At the top, it says 'Functions (10)' and 'Last fetched 7/14/2026, 10:12:33 AM'. There are buttons for 'Actions' and 'Create function'. Below this is a search bar with the placeholder text 'Search by attributes or search by keyword'. The main part of the screenshot is a table listing the functions. The table has columns for 'Function name', 'Description', 'Package type', 'Runtime', 'Type', and 'Last modified'. The functions listed are 'MainMonitoringFunction', 'getOrders', 'RedshiftOverwatch', 'RoleCreationFunction', 'getProduct', and 'addToCart'.

☐	Function name	Description	Package type	Runtime	Type	Last modified
☐	MainMonitoringFunction	-	Zip	Python 3.10	Standard	2 days ago
☐	getOrders	-	Zip	Python 3.13	Standard	2 days ago
☐	RedshiftOverwatch	Deletes Redshift Cluster if the count is more than 2.	Zip	Python 3.10	Standard	2 days ago
☐	RoleCreationFunction	Create SLR if absent	Zip	Python 3.10	Standard	2 days ago
☐	getProduct	-	Zip	Python 3.13	Standard	2 days ago
☐	addToCart	-	Zip	Python	Standard	22 hours

Figure 3.Functions In the Lambda

☐	getProduct	-	Zip	Python 3.13	Standard	2 days ago
☐	addToCart	-	Zip	Python 3.13	Standard	22 hours ago
☐	getProducts	-	Zip	Python 3.13	Standard	2 days ago
☐	RedshiftEventSubscription	Create Redshift event subscription to SNS Topic.	Zip	Python 3.10	Standard	2 days ago
☐	placeOrder	-	Zip	Python 3.13	Standard	2 days ago
☐	ModLabRole	updates LabRole to allow it to assume itself	Zip	Python 3.10	Standard	2 days ago

[Lambda](#)
>
[Functions](#)
>
Create function

Create function

Info

Choose one of the following options to create your function.

☒
Author from scratch
Start with a simple Hello World example.

☐
Use a blueprint
Build a Lambda application from sample code and configuration presets for common use cases.

☐
Container image
Select a container image to deploy for your function.

Basic information

Function name

Enter a name that describes the purpose of your function.

myFunctionName

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime

Info

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Node.js 24.x

Permissions

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution role** under **Additional settings**.

16 | Page

inventoryUpload

▼ Function overview [Info](#)

Diagram

Template



inventoryUpload



Layers

(0)



S3

+ Add trigger

Figure 4.Design of inventoryUpload Function

Code | Test | Monitor | **Configuration** | Aliases | Versions

General configuration
Triggers
Permissions
Destinations
Function URL
Environment variables

Triggers (1) [Info](#)

Fix errors Edit Delete **Add trigger**

☐ Trigger

S3: productinventory204216
arn:aws:s3:::productinventory204216
► Details

Figure 5.Triggers in inventoryUpload Function

Trigger configuration

 S3
aws asynchronous storage

Bucket
Choose or enter the ARN of an S3 bucket that serves as the event source. The bucket must be in the same region as the function.



Bucket must be in region ap-south-1

Event types
Select the events that you want to have trigger the Lambda function. You can optionally set up a prefix or suffix for an event. However, for each bucket, individual events cannot have multiple configurations with overlapping prefixes or suffixes that could match the same object key.

Prefix - optional
Enter a single optional prefix to limit the notifications to objects with keys that start with matching characters. Any [special characters](#) must be URL encoded.

Suffix - optional
Enter a single optional suffix to limit the notifications to objects with keys that end with matching characters. Any [special characters](#) must be URL encoded.

Lambda will add the necessary permissions for AWS S3 to invoke your Lambda function from this trigger. [Learn more](#) about the Lambda permissions model.

Figure 6.S3 bucket Trigger Configuration in inventoryUpload Function

Lambda > Functions > inventoryUpload > Edit basic settings

Your function is allocated CPU proportional to the memory configured.

MB
Set memory to between 128 MB and 10240 MB

Ephemeral storage [Info](#)
You can configure up to 10 GB of ephemeral storage (/tmp) for your function. [View pricing](#)
 MB
Set ephemeral storage (/tmp) to between 512 MB and 10240 MB.

SnapStart [Info](#)
Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review the [SnapStart compatibility considerations](#). For Python and .NET runtimes, [view pricing](#).

Supported runtimes: .NET 8 (C#/F#/PowerShell), Java 11, Java 17, Java 21, Python 3.12, Python 3.13.

Timeout
 min sec

Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).
☒ Use an existing role
☐ Create a new role from AWS policy templates

Existing role
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.
 
[View the InventoryLambdaExecutionRole role](#) on the IAM console.

Figure 7.inventoryUpload Basic Settings

▼ Function overview [Info](#)

Diagram

Template



lowInventory



Layers

(0)



DynamoDB

+ Add trigger

Figure 8.Design of lowInventory Function

≡ [Lambda](#) > [Functions](#) > [lowInventory](#) > Edit trigger

Edit trigger

Trigger configuration



DynamoDB

aws database event-source-mapping nosql polling

DynamoDB table

Choose or enter the ARN of a DynamoDB table.

arn:aws:dynamodb:ap-south-1:888947293269:table/AppleInventoryTable/stream/2025-07-05T03:46:52.882



Event poller configuration



Activate trigger

Select to activate the trigger now. Keep unchecked to create the trigger in a deactivated state for testing (recommended).



Enable metrics

Monitor your event source with metrics. You can view those metrics in CloudWatch console. Enabling this feature incurs additional costs. [Learn more](#)

Batch size [Info](#)

The maximum number of records in each batch to send to the function.

1000

Figure 9.Dynamodb Trigger Configuration in lowInventory Function

Ephemeral storage [Info](#)
 You can configure up to 10 GB of ephemeral storage (/tmp) for your function. [View pricing](#) [↗](#)

MB
 Set ephemeral storage (/tmp) to between 512 MB and 10240 MB.

SnapStart [Info](#)
 Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review the [SnapStart compatibility considerations](#) [↗](#). For Python and .NET runtimes, [view pricing](#) [↗](#).

Supported runtimes: .NET 8 (C#/.NET/PowerShell), Java 11, Java 17, Java 21, Python 3.12, Python 3.13.

Timeout
 min sec

Execution role
 Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#) [↗](#).

☒ Use an existing role
☐ Create a new role from AWS policy templates

Existing role
 Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

[View the LowInventoryLambdaExecutionRole role](#) [↗](#) on the IAM console.

Figure 10. lowInventory Function Basic Settings

1.4. Data Storage – Amazon DynamoDB

Product records are stored in **Amazon DynamoDB**, a fast, fully managed NoSQL database.

Each entry includes:

- ProductName (Primary Key)
- StoreRegion
- ProductCount

Tables (1) Info							
Find tables		Last updated July 14, 2026, 10:17 (UTC+5:30)		Actions		Delete	
Filter by tag		Any tag key		Filter by tag value		Any tag value	
☐	Name ▲	Status ▼	Partition key ▼	Sort key ▼	Indexes ▼	Replication Regions ▼	Deletion protection ▼
☐	Products	Active	productId (S)	-	0	0	Off

Figure 11. Dynamodb Table

Tables (1) <

Filter by tag

Any tag key ▾

Filter by tag value

Any tag value ▾

Q Find tables

< 1 > ⚙

● Products ☆

Products

Last updated

July 14, 2026, 10:18 (UTC+5:30)

🔄

☆

Actions ▾

Explore table items

< Settings Indexes Monitor Global tables Backups >

ⓘ Protect your DynamoDB table from accidental writes and deletes

Edit PITR

✕

When you turn on point-in-time recovery (PITR), DynamoDB backs up your table data automatically so that you can restore to any given second in the preceding 1 to 35 days. Additional charges apply. [Learn more](#) ↗

General information Info

Partition key

productId (String)

Sort key

-

Capacity mode

On-demand

Table status

Active

Get live item count

Figure 12. Dynamodb Table items

- Supports multiple subscribers
- Fast and reliable push-based delivery
- Customizable message formatting



Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

`arn:aws:sns:ap-south-1:888947293269:LowInventoryTopic:2632756f-5482-4a54-9648-67902fad5d2f`

If it was not your intention to subscribe, [click here to unsubscribe](#).

Figure 13.SNS Subscription

Identity and Access Management (IAM)

Search IAM

Dashboard

Access management

User groups

Users

Roles

Policies

Identity providers

Account settings

Root access management New

Roles (6) Info

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

< 1 > ⚙

☐	Role name	Trusted entities	Last activity
☐	AWSServiceRoleForSupport	AWS Service: support (Service-Linker)	-
☐	AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Linker)	-
☐	InventoryLambdaExecutionRole	AWS Service: lambda	27 minutes ago
☐	inventoryUpload-role-ul9xjl32	AWS Service: lambda	-
☐	lowInventory-role-sr25shmg	AWS Service: lambda	1 hour ago
☐	LowInventoryLambdaExecutionRole	AWS Service: lambda	30 minutes ago

Figure 14.Roles in IAM

Permissions policies (2) Info

You can attach up to 10 managed policies.

Search

Filter by Type

All types

< 1 > ⚙

☐	Policy name	Type	Attached entities
☐	AllowCloudWatchLogs	Customer inline	0
☐	LowInventoryLambdaExecutionRole	Customer managed	1

Figure 15.permission policies in LowInventoryLambdaExecutionRole

Permissions policies (1) Info

You can attach up to 10 managed policies.

Search

Filter by Type

All types

< 1 > ⚙

☐	Policy name	Type	Attached entities
☐	InventoryLambdaExecutionRole	Customer managed	1

Figure 16.permission policies in InventoryLambdaExecutionRole

Permissions defined in this policy [Info](#) [Copy](#) [Edit](#) [Summary](#) [JSON](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "s3:GetObject",
8         "s3:ListBucket"
9       ],
10      "Resource": [
11        "arn:aws:s3:::productinventory204216/*",
12        "arn:aws:s3:::productinventory204216"
13      ]
14    },
15    {
16      "Effect": "Allow",
17      "Action": [
18        "dynamodb:PutItem",
19        "dynamodb:BatchWriteItem"
20      ],
21      "Resource": "arn:aws:dynamodb:ap-south-1:888947293269:table/AppleInventoryTable"
22    }
23  ]
24 }
```

Figure 17.json code in InventoryLambdaExecutionRole policy

Permissions defined in this policy [Info](#) [Copy](#) [Edit](#) [Summary](#) [JSON](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "dynamodb:PutItem",
8         "dynamodb:UpdateItem",
9         "dynamodb:GetItem",
10        "dynamodb:Scan",
11        "dynamodb:Query"
12      ],
13      "Resource": "arn:aws:dynamodb:ap-south-1:888947293269:table/AppleInventoryTable"
14    },
15    {
16      "Effect": "Allow",
17      "Action": [
18        "dynamodb:GetRecords",
19        "dynamodb:GetShardIterator",
20        "dynamodb:DescribeStream",
21        "dynamodb:ListStreams"
22      ],
23      "Resource": "arn:aws:dynamodb:ap-south-1:888947293269:table/AppleInventoryTable/stream/*"
24    },
25    {
26      "Effect": "Allow",
27      "Action": "sns:Publish",
28      "Resource": "arn:aws:sns:ap-south-1:888947293269:LowInventoryTopic"
29    }
30  ]
31 }
```

Figure 18.json code in LowInventoryLambdaExecutionRole policy

1.5. Monitoring and Logging – Amazon CloudWatch

Amazon CloudWatch is integrated for:

- Real-time monitoring of Lambda execution
- Viewing logs for debugging and error tracing
- Setting alerts for function failures or unusual activity

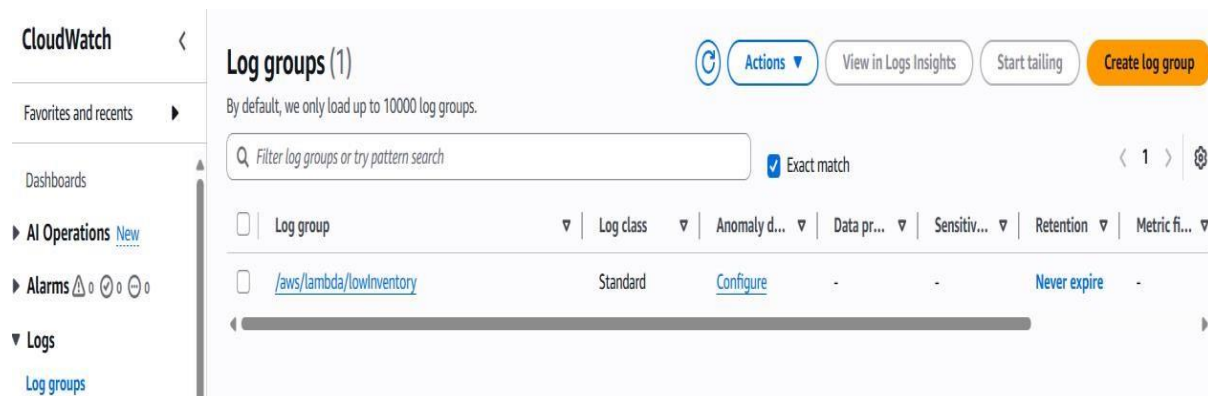


Figure 19. lowInventory function In CloudWatch

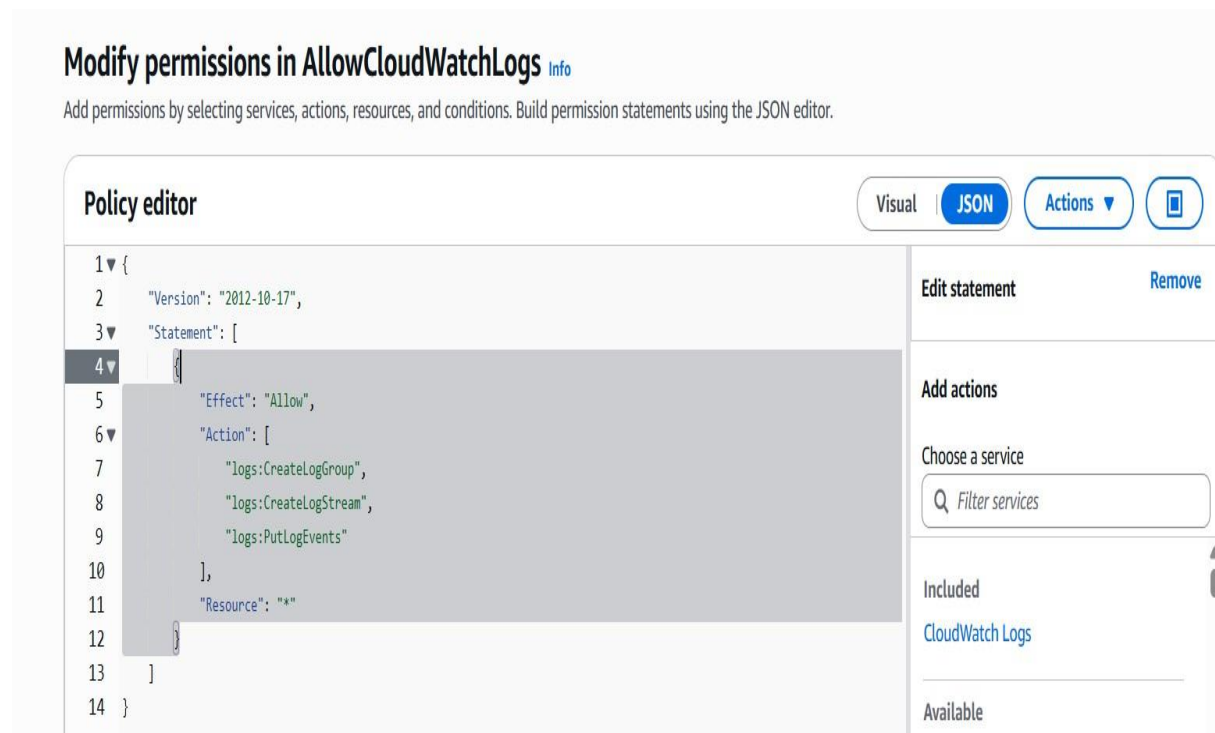


Figure 20. json inline code in CloudWatch Policy

IMPLEMENTATION

The implementation of the CloudCart Serverless E-Commerce Platform transforms the designed architecture into a complete cloud-native application using Amazon Web Services (AWS). Each AWS service was carefully configured and integrated to provide secure authentication, static website hosting, API management, backend processing, database storage, and email notifications. The system enables users to register, log in, browse products, manage shopping carts, and place orders through a responsive interface. The backend leverages AWS Lambda, API Gateway, DynamoDB, Amazon Cognito, Amazon S3, and Amazon SES to create a scalable, secure, and highly available e-commerce solution without the need to manage physical servers or traditional infrastructure.

1.6. CSV File Upload to Amazon S3

The process begins with the user uploading a .csv file containing two columns: **Product Name** and **Quantity**. An **Amazon S3 bucket** is configured to serve as the input layer for file ingestion.

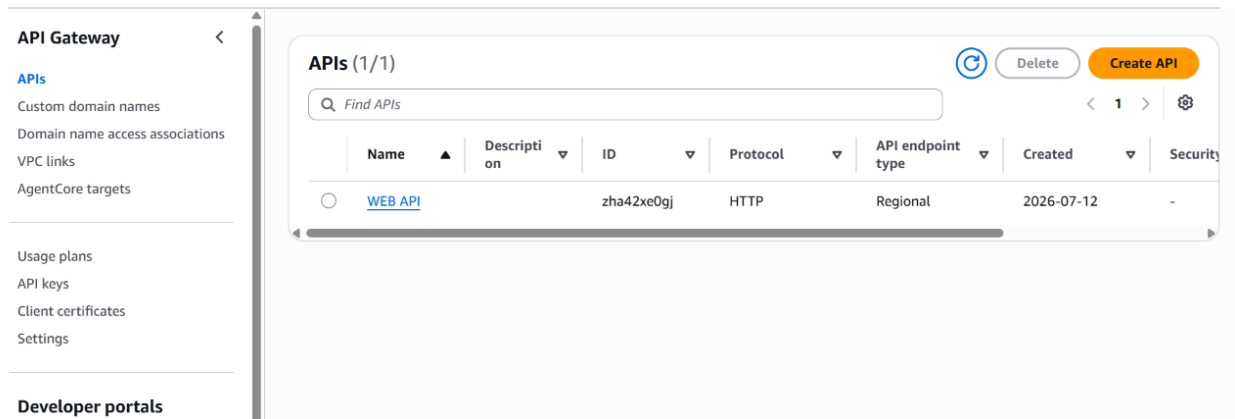
The S3 bucket has an **event notification** configured to trigger a Lambda function whenever a new file is uploaded.

1.7. AWS Lambda for File Processing and Business Logic

An **AWS Lambda function** (Python) is triggered upon the file upload. It performs several critical tasks:

1. **Reads the CSV file** from S3.
2. **Parses and validates** each row of the file.
3. **Uploads the data** (ProductName, Quantity, Timestamp) into the **DynamoDB** table.

4. **Scans** the table to check for products where the quantity is less than or equal to a threshold (e.g., 5).
5. **Sends an alert** via Amazon SNS with a list of low-quantity items.



Key Lambda details:

- Runtime: Python 3.12
- IAM role with S3 read, DynamoDB write/read, and SNS publish permissions
- Logging enabled using **Amazon CloudWatch Logs**

1.8. Data Storage with Amazon DynamoDB

A **DynamoDB table** is created to store inventory data.

- Table name: AppleInventoryTable
- Primary key: Product (string)
- Additional attributes: StoreRegion (string), ProductCount (integer)

Features:

- Automatic scaling to handle varying loads
- Real-time data access for reads and writes
- TTL or secondary indexes can be added for analytics or history tracking (future scope)

1.9. Low-Stock Notification with Amazon SNS

Once the Lambda function identifies products below the threshold, it compiles the product names and quantities into a summary message:

- This message is published to an **Amazon SNS topic**.

- Email subscription is added to the topic for real-time alert delivery.
- SNS ensures that stakeholders are notified promptly to restock items.

1.10. IAM Roles and Security

All interactions between services are secured using **IAM roles** with the principle of **least privilege**:

- Lambda role: Access to read S3, read/write DynamoDB, and publish to SNS.
- S3 bucket policy: Triggers Lambda on object creation.
- SNS topic policy: Allows publish from Lambda and subscription by verified email.

1.11. Monitoring with Amazon CloudWatch

To ensure visibility into the system's behavior and performance:

- CloudWatch logs capture Lambda execution details and errors.
- Metrics such as invocation count, execution duration, and error count are monitored.
- Optional alarms can be configured for failed Lambda executions or missed notifications.

The successful implementation of this serverless system showcases how AWS services can be integrated to create a real-time, low-maintenance, and scalable solution for inventory tracking. The modular and event-driven approach ensures flexibility for future enhancements such as API endpoints, dashboards, or predictive restocking logic.

2. RESULTS

The implementation of the CloudCart Serverless E-Commerce Platform produced successful outcomes that aligned with the project's objectives. The application was tested across multiple scenarios to validate its functionality, security, scalability, performance, and reliability. The results demonstrate the effectiveness of leveraging AWS serverless services to provide secure user authentication, seamless product browsing, efficient shopping cart management, reliable order processing, and automated backend operations. The serverless architecture ensured high availability, reduced infrastructure management, and delivered a fast, responsive, and cost-effective online shopping experience.

2.1. Automated Inventory Data Processing

When products with quantities below the defined threshold are detected, the Lambda function compiles a summary of low-stock items and sends an email alert via **Amazon SNS**. The emails are delivered to subscribed users almost instantaneously, confirming that the notification mechanism is reliable and timely.

2.2. Real-Time Low-Stock Notifications

When products with quantities below the defined threshold are detected, the Lambda function compiles a summary of low-stock items and sends an email alert via **Amazon SNS**. The emails are delivered to subscribed users almost instantaneously, confirming that the notification mechanism is reliable and timely.

2.3. Scalable and Serverless Architecture

The system handles varying volumes of data without any manual scaling or downtime, thanks to the underlying **serverless AWS infrastructure**. The combination of S3, Lambda, DynamoDB, and SNS ensures that each component functions independently and scales automatically based on demand.

Testing scenarios included:

- Uploading small and large CSV files
- Submitting multiple files in quick succession
- Monitoring invocation logs and latency

All scenarios were handled successfully without performance degradation.

2.4. Secure and Controlled Service Access

Using **AWS IAM roles and policies**, secure access was configured between all services:

- Lambda functions could only access required resources (S3, DynamoDB, SNS)
- S3 bucket events were restricted to specific Lambda triggers
- SNS topic subscriptions were managed through verified email endpoints

This validated that the system follows **best practices for cloud security and permissions**.

2.5. System Monitoring and Logging

Through **Amazon CloudWatch**, the system was monitored for:

- Lambda execution success/failure
- Processing time
- Errors and exception handling

The logs confirmed successful Lambda execution for all test cases and provided helpful debugging information during development and refinement.

2.6. User Experience and Functionality

The system requires minimal user interaction:

- Uploading a CSV file is the only manual step

Amazon Cognito

User pools

User pools

Identity pools

User pools

Provisioned limits

New

User pools (2)

Info

Refresh

Delete

Create user pool

View and configure your user pools. User pools are directories of federated and local user profiles. They provide authentication options for your users.

Search user pools by name or ID

< 1 > Settings

	User pool name	User pool ID	Regions	Created time	Last updated time
☐	User pool - dt-ug8	us-east-1_cQxXXrvqk	Single-Region	2 days ago	2 days ago
☐	User pool - p8ruho	us-east-1_4XFulbPVc	Single-Region	12 hours ago	12 hours ago

Amazon Cognito

Current user pool

View all

User pool - p8ruho

Overview

Applications

User management

Authentication

Overview: User pool - p8ruho

Info

Rename

Delete user pool

User pool information

User pool name

User pool - p8ruho

User pool ID

us-east-1_4XFulbPVc

ARN

arn:aws:cognito-idp:us-east-1:600946211300:userpool/us-east-1_4XFulbPVc

Estimated number of users

3

OpenID Connect configuration URL

https://cognito-idp.us-east-1.amazonaws.com/us-east-1_4XFulbPVc/.well-known/openid-configuration

Token signing key URL

https://cognito-idp.us-east-1.amazonaws.com/us-east-1_4XFulbPVc/.well-known/jwks.json

Feature plan

Essentials

Created time

July 13, 2026 at 22:43 GMT+5:30

Last updated time

July 13, 2026 at 22:43 GMT+5:30

Regions

US East (N. Virginia) - Primary

Current Region

Create a replica user pool

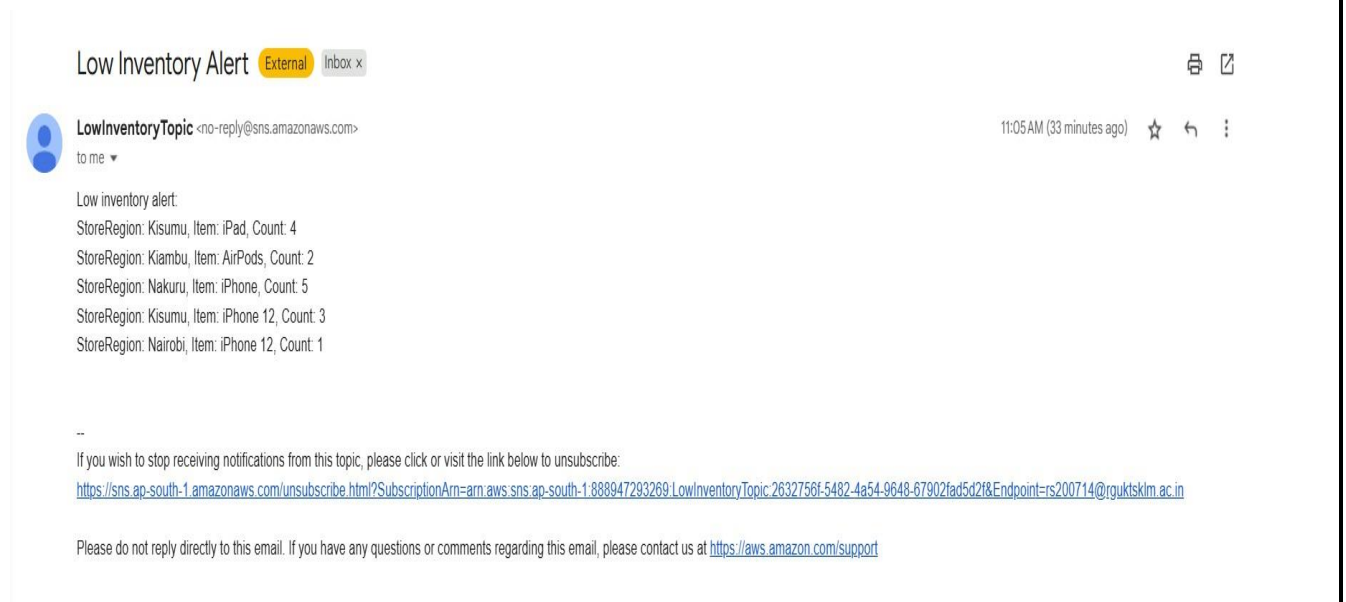
Recommendations

- All subsequent actions are fully automated
- Notifications are sent in a clear, human-readable format

This ease of use confirms the system's potential for **real-world deployment** in retail, warehouse, or inventory management environments.

The overall results demonstrate that the serverless inventory system meets its intended goals of automation, efficiency, scalability, and ease of use. The architecture is stable and ready for real-time operational use, with opportunities for further expansion and integration.

2.7. Screenshots of the output



3. CONCLUSION

The successful development and deployment of **CloudCart**, a **Serverless E-Commerce Platform**, demonstrate the effectiveness of cloud-native and serverless architectures in building modern, scalable, and secure web applications. By leveraging core AWS services such as **Amazon S3, Amazon Cognito, Amazon API Gateway, AWS Lambda, Amazon DynamoDB, and Amazon SES**, the platform successfully delivers a seamless online shopping experience without the need to manage traditional server infrastructure.

The project highlights the advantages of **serverless computing**, including automatic scaling, high availability, cost optimization, fault tolerance, and simplified infrastructure management. Amazon Cognito ensures secure user authentication and authorization, while AWS Lambda executes backend business logic on demand. Amazon DynamoDB provides fast and reliable storage for products, user accounts, shopping carts, and order information. Amazon API Gateway securely manages communication between the frontend and backend, and Amazon SES enables automated order confirmation and notification emails.

Throughout the development process, emphasis was placed on **security, performance, and scalability**. AWS IAM policies, HTTPS communication, and managed cloud services were used to secure application resources. The platform was tested under various user scenarios to verify reliable authentication, product browsing, shopping cart operations, and order processing while maintaining consistent performance.

The project also provides a strong foundation for future enhancements, including:

- **Integration of secure online payment gateways** (UPI, Credit/Debit Cards, Net Banking)
- **Real-time order tracking and delivery status updates**
- **AI-based product recommendation and personalized shopping experience**
- **Multi-vendor marketplace support**
- **Mobile application development for Android and iOS**
- **Advanced search, filtering, wishlist, and customer analytics features**

In conclusion, **CloudCart** serves as a practical, secure, and scalable blueprint for developing cloud-native e-commerce applications using AWS serverless technologies. The project demonstrates how multiple AWS managed services can be integrated to build a high-performance, reliable, and future-ready online shopping platform while minimizing operational complexity and infrastructure costs.
